

Literature Survey

Movie Recommendation System Using Collaborative Filtering and Deep Learning

1. Introduction to the Problem

A **movie recommendation system** is a software system that suggests movies to a user based on information such as the ratings the user has given in the past, the ratings given by other users, and (optionally) information about the movies themselves. Instead of asking the user to search through thousands of titles, the system predicts which titles the user is most likely to enjoy and presents a short, ordered list.

Recommendation systems are important because online catalogues have become far larger than any person can browse. Streaming platforms hold tens of thousands of titles, and a user typically sees only a few of them on a home screen. A good recommender reduces this information overload, helps users discover content they would not have searched for, and increases user engagement with the platform. Industrial systems report that a large share of what users actually watch comes from recommendations rather than from search.

Machine learning is useful here because user preference is not something we can write down as a fixed rule. We cannot say "everyone who likes movie A will like movie B." What we can do is learn this relationship from data: from millions of past ratings, a model can learn statistical patterns that connect users and movies. Machine learning also handles the scale of the problem — models can be trained once offline and then used to score many items quickly.

Collaborative filtering (CF) is the most widely used family of recommendation methods. The core idea is simple: the system uses only the user-item interaction data (ratings, clicks, watches) and assumes that users who agreed in the past will agree in the future. If two users have rated many of the same movies similarly, then a movie liked by one is a good candidate for the other. CF does not require knowing anything about the content of the movie — no genre, no cast, no plot. This is both its strength (it works for any type of item) and its weakness (it cannot say anything about an item nobody has rated yet).

Deep learning in recommendation systems means using neural networks to learn the representations of users and items and the function that combines them. Instead of representing a user by a hand-designed feature vector, we let the network learn a dense vector (an *embedding*) for each user and each movie directly from the interaction data. Instead of combining these vectors with a fixed arithmetic operation, we can pass them through several neural network layers so that the model can learn a more flexible scoring function.

Combining collaborative filtering and deep learning is attractive because the two contribute different things. Collaborative filtering supplies the learning signal — the user-item interaction matrix that carries the actual preference information. Deep learning supplies the representation power — the ability to learn embeddings automatically and to model relationships between user and item factors that are not a simple linear product. Several published models take exactly this route, and they are reviewed in Section 2.

The main challenges reported repeatedly in the literature are:

- **Data sparsity.** In a typical dataset, each user rates only a tiny fraction of available movies. In MovieLens 1M, about 95.5% of the user-item matrix is empty, and the figure is far higher in real industrial datasets. Similarity estimates computed from very few co-rated items are unreliable.
- **Cold start.** A new user has no ratings, and a new movie has no ratings. Pure collaborative filtering has no basis on which to make a prediction for either.
- **Scalability.** Computing similarities between all pairs of users becomes expensive as the number of users grows, and scoring millions of candidate items under a strict latency budget is a serious engineering problem.
- **Capturing complex user-item preferences.** Classical matrix factorization scores a user-item pair with an inner product of two latent vectors, which is a linear operation. Whether this is expressive enough to capture real preference structure has been actively debated (see Sections 2.6 and 2.10).

This survey reviews ten real research papers covering these ideas, summarises the standard datasets, identifies the research gap, and then describes our proposed approach.

2. Literature Survey

Ten papers were selected because together they cover the full path from classical neighbourhood-based collaborative filtering, through matrix factorization, to deep learning models, and finally to the recent critical work that questions how much deep models actually improve over strong classical baselines. Every fact below is taken from the cited paper itself.

2.1 Item-Based Collaborative Filtering Recommendation Algorithms

- **Authors:** B. Sarwar, G. Karypis, J. Konstan, J. Riedl
- **Year:** 2001 (WWW '01)
- **Dataset:** MovieLens — 100,000 ratings from 943 users on 1,682 movies
- **Method:** Item-based k-nearest-neighbour collaborative filtering. Similarity between item pairs is computed using cosine, correlation, or adjusted cosine similarity; predictions are produced by a weighted sum or by a regression model over similar items.
- **Input features:** The user-item rating matrix only. No movie content is used.

- **Evaluation metrics:** Mean Absolute Error (MAE); also throughput and computational cost.
- **Main result:** Item-based algorithms give lower MAE than user-based algorithms, with adjusted cosine similarity performing best among the similarity measures tested. Because item-item similarities are relatively stable, they can be precomputed offline, which makes online recommendation much faster than user-based CF.
- **Main limitation:** It is still a memory-based method that depends entirely on co-rated items. It degrades when the matrix is very sparse, and it offers no mechanism for new users or new movies.
- **Relevance to our project:** This is the classical collaborative filtering baseline. Our project will implement item-based (and user-based) CF as a comparison point against the deep model, and this paper defines how that baseline is normally built and evaluated.

2.2 Matrix Factorization Techniques for Recommender Systems

- **Authors:** Y. Koren, R. Bell, C. Volinsky
- **Year:** 2009 (IEEE *Computer*)
- **Dataset:** Netflix Prize data — over 100 million ratings from roughly 480,000 users on nearly 18,000 movie titles
- **Method:** Latent factor models based on matrix factorization. Each user and each movie is mapped to a vector of latent factors, and a rating is predicted as the inner product of the two vectors. The paper describes extensions: user and item bias terms, SVD++ (which adds implicit feedback), and temporal dynamics (factors that drift over time). Training uses stochastic gradient descent or alternating least squares with regularization.
- **Input features:** Rating matrix, plus which items a user rated (implicit signal) and rating timestamps for the temporal variants.
- **Evaluation metrics:** RMSE.
- **Main result:** Matrix factorization outperforms classical nearest-neighbour techniques on the Netflix data, and RMSE improves steadily as bias terms, implicit feedback, and temporal effects are added to the basic model. The results are presented as RMSE curves against the number of latent factors.
- **Main limitation:** The interaction is modelled by a fixed inner product, which is linear in the latent factors. The model also cannot handle cold-start users or items without additional side information.
- **Relevance to our project:** Matrix factorization is the direct ancestor of the embedding layers used in modern neural recommenders. Understanding it is necessary to explain what our embeddings represent and what a deep model adds on top of them.

2.3 AutoRec: Autoencoders Meet Collaborative Filtering

- **Authors:** S. Sedhain, A. K. Menon, S. Sanner, L. Xie
- **Year:** 2015 (WWW '15 Companion)
- **Dataset:** MovieLens 1M, MovieLens 10M, and Netflix
- **Method:** An autoencoder for collaborative filtering. Item-based AutoRec (I-AutoRec) takes an item's partially observed rating vector as input, encodes it into a low-dimensional hidden layer, and reconstructs the full rating vector; missing entries in the reconstruction are the predictions. A user-based variant (U-AutoRec) is also defined. Only observed ratings contribute to the loss.
- **Input features:** Rating vectors from the user-item matrix.
- **Evaluation metrics:** RMSE.
- **Main result:** I-AutoRec reports an RMSE of 0.831 on MovieLens 1M, and it outperforms biased matrix factorization and RBM-based collaborative filtering; on MovieLens 10M its performance is comparable to LLORMA, which combines 50 local factorization models. A deeper three-hidden-layer version with greedy pre-training further reduced RMSE on MovieLens 1M from 0.831 to 0.827.
- **Main limitation:** It targets rating prediction rather than top-N ranking, uses no side information, and does not address cold start. The paper is a two-page short paper, so the analysis is limited.
- **Relevance to our project:** It is an early and clear demonstration that a neural non-linear model can beat linear matrix factorization on movie rating data, which is the central premise of our project.

2.4 Collaborative Deep Learning for Recommender Systems

- **Authors:** H. Wang, N. Wang, D.-Y. Yeung
- **Year:** 2015 (KDD '15)
- **Dataset:** citeulike-a (5,551 users, 16,980 items), citeulike-t (7,947 users, 25,975 items), and a subset of Netflix in which movie plots are used as item content
- **Method:** Collaborative Deep Learning (CDL), a hierarchical Bayesian model that couples a Bayesian stacked denoising autoencoder (which learns a representation of item content) with probabilistic matrix factorization (which learns from the rating matrix). The two components influence each other in both directions during training.
- **Input features:** Binary implicit feedback matrix plus bag-of-words content for each item.
- **Evaluation metrics:** Recall@M and mean Average Precision (mAP).
- **Main result:** CDL outperforms collaborative topic regression, collaborative matrix factorization, SVDFeature, and a content-only deep model, with the largest gains in the sparse setting where few ratings per item are available.
- **Main limitation:** It requires item content text, uses a comparatively complex Bayesian training procedure, and its exact user/item counts for the Netflix subset are reported only inside the paper. Later reproducibility work (Section 2.9) found that simple baselines are competitive with CDL on the CiteULike datasets.
- **Relevance to our project:** It is the standard reference for *hybrid* deep recommendation — combining interaction data with content — which is the natural extension of our project once the pure collaborative model is working.

2.5 Deep Neural Networks for YouTube Recommendations

- **Authors:** P. Covington, J. Adams, E. Sargin
- **Year:** 2016 (RecSys '16)
- **Dataset:** Proprietary YouTube logs. The paper reports a vocabulary of 1 million videos and 1 million search tokens, each embedded in 256 dimensions, with models trained on hundreds of billions of examples. Exact user, item, and interaction counts are not reported in the paper.
- **Method:** A two-stage architecture. A deep *candidate generation* network treats recommendation as extreme multiclass classification over the video corpus and reduces millions of videos to hundreds; a separate deep *ranking* network then scores those candidates using weighted logistic regression trained to predict expected watch time. Both use averaged embeddings of sparse IDs fed into stacked ReLU layers.
- **Input features:** Watch history, search history, demographic features, and an "example age" feature; the ranking model adds hundreds of item and user-item features.
- **Evaluation metrics:** Offline holdout Mean Average Precision (MAP) for candidate generation; a weighted, per-user watch-time loss for ranking; live A/B tests on click-through rate and watch time for final decisions.
- **Main result:** Both depth and additional features improve holdout MAP. For the ranking model, the weighted per-user loss falls from 41.6% with no hidden layers to 34.6% for a 1024→512→256 ReLU network.
- **Main limitation:** The dataset and code are proprietary, so the results cannot be reproduced. The system also requires heavy feature engineering and large-scale infrastructure.
- **Relevance to our project:** It shows the industrial version of the same idea we are using — learned ID embeddings passed through fully connected layers — and it justifies the embedding-plus-MLP design at a scale far beyond MovieLens.

2.6 Neural Collaborative Filtering

- **Authors:** X. He, L. Liao, H. Zhang, L. Nie, X. Hu, T.-S. Chua
- **Year:** 2017 (WWW '17)
- **Dataset:** MovieLens 1M (1,000,209 interactions, 6,040 users, 3,706 items, 95.53% sparsity) and Pinterest (1,500,809 interactions, 9,916 items, 55,187 users, 99.73% sparsity)
- **Method:** The NCF framework. User and item IDs are one-hot encoded, passed through an embedding layer, and then combined by a learned function instead of a fixed inner product. Three instantiations are given: GMF (generalized matrix factorization), MLP (concatenated embeddings through hidden layers), and NeuMF, which fuses GMF and MLP with separate embeddings and a concatenated final layer. Training minimises a binary cross-entropy (log) loss with negative sampling.
- **Input features:** User ID and item ID only, with explicit ratings converted to binary implicit feedback.
- **Evaluation metrics:** Hit Ratio@10 and NDCG@10 under leave-one-out evaluation with 100 sampled negative items per user.
- **Main result:** NeuMF achieves the best performance on both datasets. On MovieLens 1M with 64 predictive factors and pre-training, it reports HR@10 = 0.730 and NDCG@10 = 0.447. The average relative improvement over the eALS and BPR baselines is 4.5% and 4.9% respectively, and stacking more MLP layers consistently improved results.
- **Main limitation:** It uses only IDs — no content and no context — so cold start is not addressed. It uses binarized implicit feedback rather than the actual rating values. Its central claim was later challenged (Section 2.10).
- **Relevance to our project:** This is the paper our proposed architecture follows most closely: embeddings for users and movies, a neural interaction function, and ranking metrics rather than accuracy.

2.7 Deep Matrix Factorization Models for Recommender Systems

- **Authors:** H.-J. Xue, X.-Y. Dai, J. Zhang, S. Huang, J. Chen
- **Year:** 2017 (IJCAI-17)
- **Dataset:** MovieLens 100K, MovieLens 1M, Amazon Music, Amazon Movies
- **Method:** Deep Matrix Factorization (DMF). A user-item interaction matrix built from explicit ratings and non-preference implicit feedback is used directly as input; a row (for a user) and a column (for an item) are projected through separate multi-layer networks into a shared low-dimensional space, and the score is the cosine similarity of the two projections. A normalized cross-entropy loss is proposed that uses the rating values rather than only binary labels.
- **Input features:** Rows and columns of the interaction matrix (explicit ratings plus implicit feedback).
- **Evaluation metrics:** HR@10 and NDCG@10.
- **Main result:** DMF-2-nce outperformed all compared methods on the four datasets. Against NeuMF-p it reports relative improvements of 2.5–7.4% (5.1% average) in NDCG and 1.4–6.8% (3.8% average) in HR.
- **Main limitation:** Using whole matrix rows and columns as input means the input dimension grows with the number of users and items, which limits scalability. Independent reproducibility work later found it was outperformed by tuned classical baselines on three of four datasets.
- **Relevance to our project:** It shows an alternative way to combine CF and deep learning — deep projection of interaction vectors rather than ID embeddings — and it is a useful contrast to the NCF design we plan to follow.

2.8 Deep Learning Based Recommender System: A Survey and New Perspectives

- **Authors:** S. Zhang, L. Yao, A. Sun, Y. Tay
- **Year:** 2019 (ACM Computing Surveys)
- **Dataset:** Not applicable — this is a survey, not an experimental paper.
- **Method:** A systematic review and taxonomy of neural recommendation models, organised by the neural building block used (MLP, autoencoder, CNN, RNN, attention, deep reinforcement learning, adversarial networks) and by whether the model is purely neural or

integrates a classical model.

- **Input features:** Not applicable.
- **Evaluation metrics:** Not applicable; the paper discusses the metrics used across the field.
- **Main result:** Deep learning models have been applied successfully across recommendation tasks, mainly because they learn non-linear interactions and can absorb heterogeneous side information. The survey also lists open problems, including explainability, efficiency, cold start, and inconsistent evaluation practice.
- **Main limitation:** As a survey it reports no new experiments, and it predates the reproducibility debate that followed.
- **Relevance to our project:** It provides the vocabulary and classification used in this report, and it supports our framing of the project as an MLP-based neural collaborative model.

2.9 Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches

- **Authors:** M. F. Dacrema, P. Cremonesi, D. Jannach
- **Year:** 2019 (RecSys '19)
- **Dataset:** The datasets used by the 18 neural papers analysed, including MovieLens and CiteULike variants
- **Method:** A systematic reproducibility study. The authors attempted to reproduce 18 neural top-N recommendation papers from major conferences and compared each against carefully tuned non-neural baselines such as item-kNN, user-kNN, graph-based methods, and popularity ranking.
- **Input features:** As in the original papers.
- **Evaluation metrics:** Precision, Recall, MAP, and NDCG at various cut-offs.
- **Main result:** Only 7 of the 18 papers could be reproduced with reasonable effort, and 6 of those 7 were outperformed by at least one simple, well-tuned baseline. The authors conclude that measured progress in neural recommendation is smaller than the published numbers suggest.
- **Main limitation:** The study covers top-N recommendation from implicit feedback and a specific set of papers; it does not claim that deep learning is useless, only that baselines are often under-tuned.
- **Relevance to our project:** It sets our evaluation standard. Our project must compare the deep model against properly tuned CF and matrix factorization baselines on identical splits, and must not claim improvement without that comparison.

2.10 Neural Collaborative Filtering vs. Matrix Factorization Revisited

- **Authors:** S. Rendle, W. Krichene, L. Zhang, J. Anderson
- **Year:** 2020 (RecSys '20)
- **Dataset:** The exact MovieLens 1M and Pinterest splits published by the NCF authors
- **Method:** A controlled re-examination of NCF. The authors compare a learned MLP similarity against a simple dot product between embeddings, using careful hyperparameter selection for both, and additionally study whether an MLP can learn to approximate a dot product.
- **Input features:** User and item IDs, as in NCF.
- **Evaluation metrics:** HR@10 and NDCG@10 on the same splits and evaluation code as NCF.
- **Main result:** With proper hyperparameter tuning, a plain dot product model outperforms the MLP-based learned similarity on all reported metrics and embedding sizes; only pre-trained NeuMF remains competitive. The paper also shows it is non-trivial for an MLP to learn a dot product, and that MLP similarities are too expensive for large-scale retrieval, where dot products allow fast nearest-neighbour search.
- **Main limitation:** The conclusion concerns the specific choice of similarity function on ID-only data; it does not evaluate deep models that use side information, sequences, or context.
- **Relevance to our project:** This is the most important cautionary paper for us. It means our project must include a tuned matrix factorization baseline and must report results honestly, rather than assuming the deep model is better because it is deeper.

3. Machine Learning and Deep Learning Techniques

3.1 Collaborative Filtering

User-based collaborative filtering. *Basic idea:* find users similar to the target user and recommend what they liked. *How it works for movies:* each user is represented by their row in the rating matrix; similarity between two users is computed with cosine similarity or Pearson correlation over co-rated movies; the predicted rating is a similarity-weighted average of the neighbours' ratings for that movie. *Advantages:* simple, intuitive, and easy to explain ("users like you also watched this"). *Limitations:* similarity must be recomputed as users rate new items, it scales poorly with the number of users, and it is unreliable when two users share very few co-rated movies.

Item-based collaborative filtering. *Basic idea:* recommend movies similar to the ones the user already liked, where similarity is defined by rating patterns and not by content. *How it works:* item-item similarities are computed from the columns of the rating matrix (adjusted cosine similarity performed best in Sarwar et al.), and the prediction for an unseen movie is a weighted sum over the user's rated movies. *Advantages:* item similarities change slowly, so they can be precomputed offline, which makes online serving fast; Sarwar et al. also reported better MAE than user-based CF. *Limitations:* still sparsity-sensitive, no cold-start handling, and the similarity table grows with the square of the catalogue size.

Matrix factorization. *Basic idea:* approximate the sparse rating matrix as the product of two low-rank matrices — a user factor matrix and a movie factor matrix. *How it works:* each user u gets a vector p_u and each movie i gets a vector q_i ; the predicted rating is their inner product, optionally plus a global mean and user/item bias terms; the factors are learned by minimising regularized squared error

on the observed ratings with stochastic gradient descent. *Advantages:* handles sparsity far better than neighbourhood methods, compresses the data into a few dozen factors, scales to the size of the Netflix dataset, and the factors often correspond to interpretable dimensions such as genre or seriousness. *Limitations:* the inner product is linear, all latent dimensions are weighted equally, and new users and movies have no factors.

3.2 Machine Learning Models Relevant to Recommendation

Beyond the CF family, two model types are directly relevant. **Regression models** (linear or ridge regression) appear inside CF itself — Sarwar et al. used a regression variant to combine similar items, and bias terms in matrix factorization are effectively a regression on user and item effects. **k-Nearest Neighbours** is the underlying algorithm of both user-based and item-based CF and remains the standard non-neural baseline; Dacrema et al. showed that a properly tuned kNN recommender is difficult for neural models to beat. Classification algorithms such as decision trees or SVMs are sometimes used for side tasks (for example, predicting whether a user will click), but they are not natural fits for the core rating-prediction problem, so they are not discussed further here.

3.3 Deep Learning Techniques

Embedding layers. *Basic idea:* map a discrete ID to a dense, trainable vector. *How it works:* user ID 231 and movie ID 1198 are each looked up in an embedding table and become, say, 32-dimensional vectors; these vectors are parameters of the network and are learned by backpropagation along with everything else. *Advantages:* they replace one-hot vectors of length equal to the number of users, learn similarity automatically (similar movies end up close together), and are exactly the neural counterpart of matrix factorization's latent factors. *Limitations:* an embedding can only be learned for an ID that appears in training, so cold start remains; embedding tables dominate the parameter count and can overfit on small datasets.

Multilayer Perceptron (MLP). *Basic idea:* stack fully connected layers with non-linear activations to learn an arbitrary function of the inputs. *How it works in a movie recommender:* concatenate the user embedding and the movie embedding, pass them through two or three hidden layers with ReLU activations (commonly halving the width at each layer), and output a predicted rating or interaction probability. *Advantages:* can represent non-linear interactions that an inner product cannot; extra features (genre, year, user demographics) can simply be concatenated to the input. *Limitations:* more hyperparameters, slower to train, and — as Rendle et al. showed — an MLP does not automatically learn even simple functions such as a dot product, and it is expensive at serving time because scores cannot be retrieved by fast nearest-neighbour search.

Neural Collaborative Filtering (NCF). *Basic idea:* replace the fixed inner product of matrix factorization with a function learned from the data. *How it works:* the GMF branch takes the element-wise product of the user and movie embeddings; the MLP branch takes their concatenation through hidden layers; NeuMF concatenates the outputs of both branches (with separate embeddings) and passes them to a single output unit with a sigmoid activation. *Advantages:* keeps the linear modelling ability of MF while adding non-linearity; a general framework, since MF is a special case of it. *Limitations:* trained on binarized implicit feedback in the original paper, uses no side information, and its advantage over a well-tuned dot product is contested.

Autoencoders. *Basic idea:* compress an input into a hidden representation and reconstruct it; the reconstruction fills in the missing entries. *How it works:* I-AutoRec feeds a movie's rating column into an encoder, obtains a compact hidden vector, and reconstructs the full column; the reconstructed values at unrated positions become predictions. *Advantages:* few parameters, fast to train, and it captures non-linear structure in ratings. *Limitations:* it optimises reconstruction of ratings rather than ranking quality, and it uses no side information.

Hybrid deep-learning approaches. *Basic idea:* learn from interaction data and content data at the same time. *How it works:* CDL learns item representations from text (plots, abstracts) with a stacked denoising autoencoder while simultaneously factorizing the rating matrix, with the two parts regularising each other. Industrial systems such as the YouTube model do the same thing more directly by concatenating watch-history embeddings with demographic and freshness features. *Advantages:* this is the main practical route to reducing cold-start and sparsity problems, since a new movie still has a plot and a genre. *Limitations:* it requires good content data, adds substantial model complexity, and increases the risk of overfitting on small datasets.

4. Datasets

The MovieLens datasets are maintained by GroupLens Research at the University of Minnesota and are the standard benchmark in this area; their history and correct use are documented by Harper and Konstan. All statistics below come from the official GroupLens dataset documentation.

Property	MovieLens 100K	MovieLens 1M	MovieLens 20M
Source	GroupLens (grouplens.org/datasets/movielens)	GroupLens	GroupLens
Users	943	6,040	138,493
Movies	1,682	approx. 3,900	27,278
Ratings	100,000	1,000,209	20,000,263
Rating scale	1-5, whole stars	1-5, whole stars	0.5-5.0, half stars
Released	April 1998	February 2003	April 2015 (updated Oct 2016)
Extra features	User demographics (age, gender, occupation, zip), movie titles, release dates, 19 genre flags, timestamps	User demographics, movie titles, genres, timestamps	Movie titles, genres, timestamps, 465,564 tag applications, tag genome relevance scores; no demographics

In every case, only users with at least 20 ratings are included. In MovieLens 1M the users are those who joined MovieLens in 2000; in MovieLens 20M users were selected at random and each is represented only by an anonymous ID.

Possible use in our project. MovieLens 100K is small enough to iterate quickly on, so it is suitable for building the pipeline and

debugging the baselines. MovieLens 1M is the main dataset used in the neural collaborative filtering literature (He et al., Sedhain et al., Xue et al., Rendle et al. all report on it), so using it makes our numbers directly comparable to published work, and it is our primary choice. MovieLens 20M would only be used if we want to test scalability, since training on 20 million ratings requires significantly more compute. Genres and timestamps are available in all three and can be used later for a hybrid extension.

The **Netflix Prize dataset** also appears repeatedly in the literature (Koren et al., Sedhain et al., Wang et al.). Its training set contains 100,480,507 ratings from 480,189 users on 17,770 movies. It is much larger than MovieLens and is not distributed as openly, so we do not plan to use it, but the RMSE values reported on it are the standard historical reference points for rating prediction.

5. Comparison of Existing Approaches

Paper	Year	Dataset	Method	Evaluation Metric	Result	Limitation
Sarwar et al.	2001	MovieLens (943 users, 1,682 movies, 100K ratings)	Item-based kNN CF	MAE	Item-based CF gives lower MAE than user-based CF; adjusted cosine is the best similarity measure	Sparsity-sensitive; no cold-start handling
Koren et al.	2009	Netflix Prize (100M+ ratings)	Matrix factorization, SVD++, temporal MF	RMSE	MF beats nearest-neighbour methods; RMSE improves as biases, implicit feedback and temporal effects are added	Linear inner product; no cold start
Sedhain et al.	2015	MovieLens 1M / 10M, Netflix	I-AutoRec autoencoder	RMSE	RMSE 0.831 on ML-1M; 0.827 with a deep 3-layer version; beats biased MF and RBM-CF	Rating prediction only; no side information
Wang et al.	2015	citeulike-a, citeulike-t, Netflix subset	Collaborative Deep Learning (SDAE + PMF)	Recall@M, mAP	Outperforms CTR, CMF, SVDFeature; largest gains in the sparse setting	Needs item content; complex training; later found beatable by simple baselines
Covington et al.	2016	Proprietary YouTube logs	Deep candidate generation + deep ranking	Holdout MAP; weighted per-user loss; live A/B	Per-user loss falls from 41.6% (no hidden layer) to 34.6% (1024→512→256)	Not reproducible; heavy feature engineering
He et al.	2017	MovieLens 1M, Pinterest	NCF: GMF, MLP, NeuMF	HR@10, NDCG@10	NeuMF HR@10 = 0.730, NDCG@10 = 0.447 on ML-1M; +4.5% over eALS, +4.9% over BPR on average	ID-only; binarized feedback; claim later contested
Xue et al.	2017	ML-100K, ML-1M, Amazon Music, Amazon Movies	Deep Matrix Factorization	HR@10, NDCG@10	DMF-2-nce improves over NeuMF-p by 5.1% (NDCG) and 3.8% (HR) on average	Input size grows with catalogue; reproducibility questioned
Zhang et al.	2019	Not applicable (survey)	Taxonomy of deep recommenders	Not applicable	Deep models are widely applicable; open issues include cold start, efficiency, explainability	No new experiments
Dacrema et al.	2019	Datasets of 18 reviewed papers	Reproducibility study vs tuned baselines	Precision, Recall, MAP, NDCG	Only 7 of 18 papers reproducible; 6 of those beaten by simple tuned baselines	Scoped to top-N implicit feedback
Rendle et al.	2020	NCF's ML-1M and Pinterest splits	Dot product vs learned MLP similarity	HR@10, NDCG@10	A tuned dot product outperforms the MLP similarity across metrics and embedding sizes	Studies ID-only similarity, not side-information models

Because the papers optimise different tasks (rating prediction versus top-N ranking), their metrics are deliberately left in their original form. An RMSE of 0.831 and an HR@10 of 0.730 measure different things and must not be compared to each other.

6. Limitations of Existing Systems

- **Data sparsity.** Every collaborative method degrades when the rating matrix is mostly empty. This is stated as a motivation in Sarwar et al., Wang et al., and the NCF paper, and MovieLens 1M is already 95.5% empty.
- **Cold start.** None of the pure collaborative methods reviewed (item-based CF, MF, AutoRec, NCF, DMF) can score a user or a movie with no interaction history. Only the hybrid and industrial systems address this, and they do so by bringing in content or demographic features.
- **Scalability and computational cost.** Neighbourhood methods scale poorly with the number of users; Rendle et al. show that MLP-based scoring cannot use fast nearest-neighbour retrieval and is therefore too costly for production; the YouTube paper describes explicit trade-offs between model depth and CPU budget.

- **Limited expressiveness of linear models.** The inner product weights all latent dimensions equally and combines them linearly, which He et al. argue is insufficient for complex preference structure.
 - **Popularity bias.** Popular movies dominate recommendations. The YouTube paper reports that a non-personalized popularity ranking is a meaningful baseline, and Dacrema et al. found that simple popularity and kNN methods sometimes match neural models.
 - **Dependence on large datasets and on evaluation choices.** Deep models have many parameters and need substantial data. Both Dacrema et al. and Rendle et al. show that results depend heavily on hyperparameter tuning, baseline strength, and the exact evaluation protocol.
 - **Lack of explainability.** Zhang et al. list explainability as an open problem: an embedding dimension has no inherent meaning, so it is hard to tell the user *why* a movie was recommended.
 - **Difficulty combining different kinds of movie information.** Genres, plots, cast, and tags come in different formats. CDL and the YouTube system show it is possible to combine them, but at a significant cost in model complexity.
-

7. Research Gap

Reading the ten papers together, several gaps become visible, and they map directly onto our project.

Traditional collaborative filtering struggles with sparse interactions. Item-based and user-based CF depend on co-rated items. When two users share only two or three movies, the similarity estimate is essentially noise. This is a structural weakness of neighbourhood methods, not a tuning problem, and it is one reason model-based methods replaced them.

Matrix factorization may not capture all complex non-linear relationships. Koren et al. themselves extend the basic inner product with biases, implicit feedback, and time, which suggests the plain bilinear form is not enough. He et al. give an explicit geometric example where a low-dimensional inner product cannot preserve the correct ranking of user similarities. Whether that theoretical limitation matters in practice is exactly what is still unresolved.

Deep learning can learn richer representations, but the evidence is contested. AutoRec, NCF, and DMF all report improvements from non-linear models. Dacrema et al. and Rendle et al. then show that some of those improvements shrink or disappear once baselines are properly tuned. This is the sharpest gap in the current literature: the field has many published gains but comparatively few carefully controlled comparisons on identical splits.

A combined approach may improve representation learning relative to classical CF alone. Learned embeddings plus a neural interaction function generalise matrix factorization rather than replacing it — MF is a special case of NCF. So a combined model can in principle be at least as good, provided it is trained and tuned as carefully as the baseline.

Cold start and sparse data remain unsolved. None of the ID-only models addresses them, and the hybrid models that do (CDL) require content data and considerable extra machinery.

The gap this project addresses. There is room for a careful, reproducible student-scale study on MovieLens that (a) implements collaborative filtering baselines and a matrix factorization baseline properly rather than as weak strawmen, (b) implements an embedding-based deep model in the NCF style, (c) evaluates all of them on identical splits with both rating-error and ranking metrics, and (d) reports whatever the outcome is — including the possibility that the tuned baseline wins, which is what Rendle et al. found. We do **not** claim in advance that our approach solves sparsity or cold start, and we do not claim it will be more accurate than existing methods; those questions can only be answered after the experiments are run.

8. Proposed Approach

Our proposed system is a movie recommendation model that learns user and movie embeddings from collaborative filtering data and combines them with a deep neural network. The pipeline is as follows.

1. **Collect movie and user-rating data.** Use a MovieLens dataset (100K for development, 1M for the main experiments), which provides user IDs, movie IDs, ratings, timestamps, titles, and genres.
2. **Preprocess the dataset.** Remove duplicates, re-index user and movie IDs to a contiguous range starting at zero (required for embedding lookup), decide whether to keep the ratings as explicit values or convert them to implicit binary signals, and split into training, validation, and test sets. The split protocol must be fixed and documented, because published results depend on it.
3. **Create user and movie representations.** Build the sparse user-item interaction matrix, and compute basic statistics (rating counts per user and per movie, global mean, sparsity) that are needed both for the baselines and for reporting.
4. **Apply collaborative filtering concepts.** Implement user-based and item-based kNN CF with cosine or adjusted cosine similarity, and a matrix factorization model with bias terms. These are our baselines, and following Dacrema et al. they will be tuned properly rather than run at default settings.
5. **Generate user and movie embeddings.** Add embedding layers that map each user ID and each movie ID to a dense vector of a chosen dimension (typically 16–64). These embeddings are randomly initialised and learned during training.
6. **Use a deep learning model to learn user-item relationships.** Follow the NCF design: a generalized matrix factorization branch that takes the element-wise product of the embeddings, an MLP branch that takes their concatenation through two or three ReLU layers, and a fused output layer. The output is a predicted rating (regression, trained with mean squared error) or an interaction probability (trained with binary cross-entropy and negative sampling), depending on which task we evaluate.
7. **Train and validate the model.** Train with Adam, use the validation set for early stopping and hyperparameter selection (embedding size, number of layers, learning rate, dropout, regularization), and never tune on the test set.
8. **Generate movie recommendations.** For a given user, score the candidate movies the user has not seen, sort by score, and return

the top-K list.

9. **Evaluate the recommendation system.** Report RMSE and MAE for rating prediction, and Precision@K, Recall@K, NDCG@K, and Hit Rate@K for the ranking task, for every model on the same split.

Why this approach suits the project. It is directly grounded in published work, so every component can be justified during a viva. It is computationally feasible on a student machine or a free GPU notebook, since MovieLens 1M is small by modern standards. It naturally includes strong baselines, which makes the study meaningful whichever model wins. And it has a clean extension path: genres and timestamps are already in the dataset, so a hybrid version in the style of CDL or the YouTube model can be added later without redesigning the pipeline.

Because the project has not been run yet, **no performance figures are claimed at this stage.** All numbers quoted in this survey belong to the cited papers.

9. How Our Project Differs from Existing Work

Part of the project as planned:

- It combines collaborative filtering with deep learning in a single pipeline, rather than studying either in isolation.
- It uses learned user and movie embeddings instead of hand-computed similarity scores.
- It attempts to capture non-linear user-item relationships through an MLP-based interaction function.
- It produces personalized top-K movie recommendations for individual users.
- It evaluates using recommendation-specific metrics (both rating-error and ranking metrics) on a fixed, documented split.
- It explicitly compares against tuned CF and matrix factorization baselines, which the reproducibility literature identifies as the main weakness in published comparisons.

Possible future improvements, not part of the current project:

- Adding movie content (genres, tags, plot text) for a hybrid model in the style of CDL.
- Handling cold start using content or demographic features.
- Modelling temporal dynamics using timestamps.
- Adding attention or sequence models over the user's watch history.
- Adding an explainability layer that tells the user why a movie was recommended.
- Scaling to MovieLens 20M or deploying the model as a web service.

We are not proposing a novel algorithm. The contribution is a careful, reproducible implementation and comparison at project scale, which is a legitimate and honest positioning for an undergraduate ML project.

10. Expected Evaluation Metrics

Recommendation quality is **not** the same as classification accuracy. A recommender is not asked to label items correctly; it is asked either to predict a rating close to the true one, or to place relevant items near the top of a short list. The metrics below reflect those two different goals.

RMSE (Root Mean Squared Error). The square root of the mean squared difference between predicted and actual ratings. Because errors are squared, large mistakes are penalised heavily. Useful when the system outputs a numeric rating on a 1–5 scale; this is the metric of the Netflix Prize and of AutoRec.

MAE (Mean Absolute Error). The mean absolute difference between predicted and actual ratings. It treats all errors proportionally and is easier to interpret ("on average we are off by 0.7 stars"). Used by Sarwar et al. Report it alongside RMSE, not instead of it.

Precision@K. Of the K movies recommended, the fraction that are actually relevant. Useful when the user only looks at the first few results and false positives are costly.

Recall@K. Of all the movies that are relevant to the user, the fraction that appear in the top K. Useful when coverage matters — whether the system finds the items the user would have liked.

F1@K. The harmonic mean of Precision@K and Recall@K. Useful as a single summary number when precision and recall must be balanced.

NDCG@K (Normalized Discounted Cumulative Gain). A ranking metric that rewards relevant items appearing higher in the list, using a logarithmic position discount, normalized so the ideal ranking scores 1. Useful because position matters: a hit at rank 1 is worth more than a hit at rank 10. Used by He et al., Xue et al., and Rendle et al.

Hit Rate@K (HR@K). The fraction of test users for whom the held-out item appears anywhere in the top K. It is a coarser measure than NDCG (it ignores position) and is standard in leave-one-out evaluation, as in NCF.

A practical note: HR and NDCG values are strongly affected by the evaluation protocol. NCF ranks the held-out item against 100 sampled negatives, so its numbers are not comparable to a system that ranks against the entire catalogue. Our project will state the protocol explicitly when reporting results.

11. Final Literature Survey Summary

Previous researchers have moved recommendation from simple neighbourhood methods to learned latent representations and then to neural models. Sarwar et al. established item-based collaborative filtering as an accurate and scalable classical approach. Koren, Bell, and Volinsky established matrix factorization as the dominant model-based method during the Netflix Prize, showing that latent factors with bias terms, implicit feedback, and temporal dynamics steadily reduce RMSE. Sedhain et al. showed that a compact autoencoder can beat linear factorization on MovieLens rating prediction. Wang et al. showed that content representation learning and collaborative filtering can be trained jointly to help in sparse settings. Covington et al. demonstrated the approach working at industrial scale with embeddings and deep ranking networks. He et al. proposed the NCF framework that generalises matrix factorization by replacing the inner product with a learned function, and Xue et al. offered a deep projection alternative. Zhang et al. organised this work into a taxonomy and named the open problems.

The major methods are therefore: neighbourhood-based collaborative filtering, matrix factorization and its extensions, autoencoder-based collaborative filtering, embedding-plus-MLP neural collaborative filtering, and hybrid models that add content.

Their main limitations are consistent across the literature: sparsity, cold start, scalability and serving cost, popularity bias, weak explainability, and — importantly — fragile experimental evidence. Dacrema et al. found most reviewed neural recommenders could not be reproduced or were beaten by tuned classical baselines, and Rendle et al. found that a well-tuned dot product outperforms the MLP similarity on the very datasets that popularised neural collaborative filtering.

The research gap this leads to is therefore not "invent a deeper network." It is that combining collaborative filtering signals with learned embeddings is theoretically well-motivated but empirically under-verified at the level of careful, like-for-like comparison, and that cold start and sparsity remain open.

Combining collaborative filtering with deep learning is still a reasonable direction for our project. Collaborative filtering supplies the preference signal, embeddings give a learned representation of users and movies, and a neural interaction function generalises matrix factorization rather than discarding it — matrix factorization is a special case of the framework we are using, so the deep model has the capacity to match it. The approach is well documented, reproducible on MovieLens with modest compute, and extends naturally to hybrid models later. What we cannot claim in advance is that it will perform better; that will be decided by the experiments, measured against properly tuned baselines and reported honestly.

12. References (IEEE Format)

- [1] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, "Item-based collaborative filtering recommendation algorithms," in *Proc. 10th Int. Conf. World Wide Web (WWW '01)*, Hong Kong, 2001, pp. 285–295. doi: 10.1145/371920.372071. Available: <https://doi.org/10.1145/371920.372071>
- [2] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, no. 8, pp. 30–37, Aug. 2009. doi: 10.1109/MC.2009.263. Available: <https://doi.org/10.1109/MC.2009.263>
- [3] S. Sedhain, A. K. Menon, S. Sanner, and L. Xie, "AutoRec: Autoencoders meet collaborative filtering," in *Proc. 24th Int. Conf. World Wide Web (WWW '15 Companion)*, Florence, Italy, 2015, pp. 111–112. doi: 10.1145/2740908.2742726. Available: <https://doi.org/10.1145/2740908.2742726>
- [4] H. Wang, N. Wang, and D.-Y. Yeung, "Collaborative deep learning for recommender systems," in *Proc. 21th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD '15)*, Sydney, Australia, 2015, pp. 1235–1244. doi: 10.1145/2783258.2783273. Available: <https://doi.org/10.1145/2783258.2783273>
- [5] P. Covington, J. Adams, and E. Sargin, "Deep neural networks for YouTube recommendations," in *Proc. 10th ACM Conf. Recommender Systems (RecSys '16)*, Boston, MA, USA, 2016, pp. 191–198. doi: 10.1145/2959100.2959190. Available: <https://doi.org/10.1145/2959100.2959190>
- [6] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural collaborative filtering," in *Proc. 26th Int. Conf. World Wide Web (WWW '17)*, Perth, Australia, 2017, pp. 173–182. doi: 10.1145/3038912.3052569. Available: <https://doi.org/10.1145/3038912.3052569>
- [7] H.-J. Xue, X.-Y. Dai, J. Zhang, S. Huang, and J. Chen, "Deep matrix factorization models for recommender systems," in *Proc. 26th Int. Joint Conf. Artificial Intelligence (IJCAI-17)*, Melbourne, Australia, 2017, pp. 3203–3209. doi: 10.24963/ijcai.2017/447. Available: <https://www.ijcai.org/proceedings/2017/447>
- [8] S. Zhang, L. Yao, A. Sun, and Y. Tay, "Deep learning based recommender system: A survey and new perspectives," *ACM Computing Surveys*, vol. 52, no. 1, Art. no. 5, pp. 1–38, Feb. 2019. doi: 10.1145/3285029. Available: <https://doi.org/10.1145/3285029>
- [9] M. F. Dacrema, P. Cremonesi, and D. Jannach, "Are we really making much progress? A worrying analysis of recent neural recommendation approaches," in *Proc. 13th ACM Conf. Recommender Systems (RecSys '19)*, Copenhagen, Denmark, 2019, pp. 101–109. doi: 10.1145/3298689.3347058. Available: <https://doi.org/10.1145/3298689.3347058>
- [10] S. Rendle, W. Krichene, L. Zhang, and J. Anderson, "Neural collaborative filtering vs. matrix factorization revisited," in *Proc. 14th ACM Conf. Recommender Systems (RecSys '20)*, Virtual Event, Brazil, 2020, pp. 240–248. doi: 10.1145/3383313.3412488. Available: <https://doi.org/10.1145/3383313.3412488>
- [11] F. M. Harper and J. A. Konstan, "The MovieLens datasets: History and context," *ACM Transactions on Interactive Intelligent Systems*, vol. 5, no. 4, Art. no. 19, pp. 1–19, Dec. 2015. doi: 10.1145/2827872. Available: <https://doi.org/10.1145/2827872>
- [12] GroupLens Research, "MovieLens datasets," University of Minnesota. Available: <https://grouplens.org/datasets/movielens/>

Note on verification

All ten reviewed papers are real, peer-reviewed publications, and every dataset statistic, metric, and numerical result quoted above is taken from the paper or the official dataset documentation cited next to it. Where a paper does not report a detail (for example, exact

user and item counts for the proprietary YouTube data), this is stated rather than filled in. No result is attributed to our own system, because the system has not yet been evaluated.